

00 - Java desde cero, paso a paso

Trece pasos para repasar lo básico de Java **antes** de empezar con objetos: el `main`, imprimir, variables, leer del teclado, decidir, repetir, arreglos y métodos. Después, tus propias clases con métodos `static`, validaciones que no dejan caer el programa y sobrecarga, la primera forma de polimorfismo. Al final se junta todo en un programa de verdad.

Cómo se trabaja con esta guía

1. **Escribe el código tú.** No copies y pegues. Los errores de tipeo son parte de la clase: cada uno te enseña a leer un mensaje de `javac`.
2. **Compila seguido.** Después de cada bloque: `javac` y `java`. Nunca escribas 40 líneas sin compilar, porque después no sabes cuál de todas falló.
3. **Cada paso es un archivo nuevo**, con su propio `main`.
4. **Las secciones "Rómpelo" no son opcionales.** Provocar el error a propósito es la única forma de reconocerlo cuando te aparezca sin querer.
5. **Si te pierdes**, la versión terminada de cada paso está en su carpeta `pasoNN/referencia/`. Compárala con la tuya, línea por línea. No la copies.

Trabaja en una carpeta **tuya**, no dentro de `pasoNN/referencia/`: ahí están las versiones terminadas, con los mismos nombres de archivo.

Cómo se leen los bloques

Cada paso empieza diciendo **qué archivo crear**. Después vienen bloques numerados (2.1, 2.2, 2.3...) y cada uno se lee así:

Bloque	Qué tienes que hacer
Código Java sin más aviso	Escribirlo dentro del <code>main</code> , debajo de lo anterior . Los bloques se van sumando: no borres lo de antes.
Código con un aviso arriba ("borra...", "reemplaza...", "arriba del <code>main</code> ")	Lo que diga el aviso. Cuando no dice nada, es "debajo de lo anterior".
Un bloque sin <code>java</code> arriba, después del código	Es la salida esperada : lo que tiene que aparecer en tu pantalla. En los programas que preguntan cosas, lo que se ve después de la pregunta es lo que escribes tú .
Rómpelo	Provocar el error a propósito, leer el mensaje y deshacer el cambio antes de seguir.

Bloque	Qué tienes que hacer
Ahora tú	Dos o tres desafíos cortos. Los desarrollados, con su salida esperada, están en la hoja de ejercicios del paso.

Al final de cada paso hay un recuadro con el archivo terminado, dentro de `referencia/`, para comparar.

Paso 0 - Antes de empezar

Abre una terminal y comprueba que tienes Java:

```
java -version
javac -version
```

Los dos tienen que responder con un número, 17 o mayor.

Si `javac` no se reconoce pero `java` sí, tienes instalado solo el JRE (el que *ejecuta*) y te falta el JDK (el que *compila*), o el JDK no está en el PATH. Instala el JDK 17 (o uno más nuevo) para tu sistema operativo y vuelve a probar los dos comandos antes de seguir.

Crea tu carpeta de trabajo y entra en ella:

```
mkdir java-base
cd java-base
```

¿Usas IntelliJ, NetBeans o VS Code? El botón ► hace el `javac` y el `java` por ti. Igual haz el paso 1 en la terminal al menos una vez: es la única forma de ver qué hace ese botón.

Paso 1 - El primer programa

Objetivo: escribir, compilar y ejecutar un programa. Entender qué es cada palabra de las primeras líneas.

1.1 Escribe

Crea un archivo llamado **exactamente** `Paso01Hola.java` y escribe:

```
public class Paso01Hola {
    public static void main(String[] args) {
        System.out.println("Hola, DUOC");
        System.out.println("Mi primer programa en Java");
    }
}
```

1.2 Compila y ejecuta

```
javac Paso01Hola.java      # compila: crea Paso01Hola.class
java Paso01Hola           # ejecuta la CLASE, sin extension
```

Deberías ver:

```
Hola, DUOC
Mi primer programa en Java
```

Haz `ls` (o `dir` en Windows): apareció un archivo `Paso01Hola.class`. Eso es lo que produjo `javac`, y es lo que ejecuta `java`.

1.3 Qué es cada cosa

Pedazo	Qué significa
<code>public class Paso01Hola</code>	En Java todo vive dentro de una clase. El archivo se llama igual que ella.
<code>public static void main(String[] args)</code>	El punto de partida. <code>java</code> busca un método con exactamente esta forma.
<code>{ }</code>	Abren y cierran bloques. Cada <code>{</code> tiene su <code>}</code> .
<code>System.out.println(...)</code>	Imprime y salta de línea.
<code>;</code>	Termina cada instrucción. Es obligatorio.

Por ahora el `main` se escribe de memoria. En el paso 9 vas a poder leer cada una de sus palabras.

Rómpelo

Haz cada cambio, compila, lee el error, y **deshazlo** antes del siguiente.

Cambio	Lo que dice <code>javac</code>
Borra el <code>;</code> del final del primer <code>System.out.println</code>	<code>error: ';' expected</code>
Escribe <code>system</code> con minúscula	<code>error: package system does not exist</code>
Borra la <code>"</code> del final de <code>"Hola, DUOC"</code>	<code>error: unclosed string literal</code>
Renombra la clase a <code>Hola</code> (el archivo no)	<code>error: class Hola is public, should be declared in a file named Hola.java</code>

Lee el número de línea. Cada error empieza con algo como `Paso01Hola.java:3:`. Ese 3 es la línea donde `javac` se dio cuenta, que casi siempre es donde está el error o la de justo antes.

Si vienes de Python

```
print("Hola, DUOC")
```

Una línea. En Java son cinco, y ninguna es opcional: la clase, el `main`, las llaves, el `;`. A cambio, Java revisa todo **antes** de ejecutar: el error del `;` apareció sin que el programa corriera ni una línea.

Ahora tú

- Agrega una tercera línea que imprima tu nombre y tu carrera.

✓ **Al terminar el paso 1**, tu código tiene que hacer lo mismo que `Paso01Hola.java`. Compruébalo con `javac Paso01Hola.java` y `java Paso01Hola`, y si algo no calza, compara los dos archivos línea por línea.

 **Ejercicios del paso 1:** [paso01/ejercicios.md](#)

Paso 2 - Imprimir

Objetivo: controlar lo que sale por pantalla: saltos de línea, textos con números y el formato de los decimales.

Crea `Paso02Imprimir.java` con el esqueleto de siempre:

```
public class Paso02Imprimir {
    public static void main(String[] args) {

    }
}
```

Desde ahora, todo lo que escribas va **dentro del** `main`, un bloque debajo del otro. Compila y ejecuta después de cada bloque.

2.1 `println` VS `print`

```
System.out.println("Linea uno");
System.out.print("Esto queda ");
System.out.print("en la misma linea");
System.out.println();           // solo un salto de linea
```

```
Linea uno
Esto queda en la misma linea
```

`println` salta de línea al **final**. `print` no salta nunca.

2.2 Pegar textos y números con `+`

```
System.out.println("Tengo " + 20 + " años");
System.out.println("1 + 2 = " + 1 + 2);
System.out.println("1 + 2 = " + (1 + 2));
```

```
Tengo 20 años
1 + 2 = 12
1 + 2 = 3
```

¿Por qué la segunda línea dice 12? El `+` se lee de izquierda a derecha. `"1 + 2 = " + 1` da el texto `"1 + 2 = 1"`, y a un texto se le **pega** el 2. Los paréntesis obligan a sumar primero.

Escribimos `años` y no `años` a propósito: según cómo esté configurado el PC, la `ñ` y los tildes pueden salir como `Ã±` en la terminal. En los textos de los programas los evitamos; en los comentarios da lo mismo.

2.3 Caracteres especiales

```
System.out.println("Nombre:\tAna\nCarrera:\tInformatica");
System.out.println("Ella dijo \"hola\"");
```

```
Nombre:  Ana
Carrera: Informatica
Ella dijo "hola"
```

Se escribe	Produce
<code>\n</code>	salto de línea
<code>\t</code>	tabulación
<code>\"</code>	una comilla, sin cerrar el texto
<code>\\</code>	una barra invertida

2.4 Comentarios

```
// Esto es un comentario de una linea. Java lo ignora.

/*
 * Esto es un comentario
 * de varias lineas.
 */
```

2.5 `printf`: un molde con huecos

```
System.out.printf("%s tiene %d anios y mide %.2f m%n", "Ana", 20, 1.6789);
```

```
Ana tiene 20 anios y mide 1.68 m
```

Los huecos se rellenan **en orden** con los valores que vienen después de la coma:

Hueco	Se rellena con
<code>%s</code>	un texto (<i>string</i>)
<code>%d</code>	un entero

Hueco	Se rellena con
<code>%.2f</code>	un decimal, con 2 cifras (redondea)
<code>%n</code>	un salto de línea

`printf` **no** salta de línea solo. Si te olvidas del `%n`, lo siguiente que imprimas queda pegado.

Rómpelo

Cambio	Qué pasa
En el <code>printf</code> , cambia <code>20</code> por <code>"20"</code>	Compila, pero al ejecutar: <code>IllegalFormatConversionException: d != java.lang.String</code>
Borra el <code>1.6789</code>	Compila, pero al ejecutar: <code>MissingFormatArgumentException: Format specifier '%.2f'</code>

Estos dos **compilan**. `javac` no revisa los huecos del `printf`; el error recién aparece cuando el programa corre. Guarda esa idea para la demo 10.

Si vienes de Python

Python	Java
<code>print("hola")</code>	<code>System.out.println("hola");</code>
<code>print("hola", end="")</code>	<code>System.out.print("hola");</code>
<code>f"{nombre} tiene {edad}"</code>	<code>nombre + " tiene " + edad</code>
<code>f"{x:.2f}"</code>	<code>System.out.printf("%.2f", x);</code>
<code># comentario</code>	<code>// comentario</code>

Ojo con una diferencia: en Python `"Tengo " + 20` es un **error**. En Java funciona, porque cualquier cosa sumada a un texto se convierte en texto.

Ahora tú

- Sin ejecutarlo, adivina qué imprime cada una. Después compruébalo:

```
System.out.println("" + 1 + 2);
System.out.println(1 + 2 + "");
```

- Imprime esta boleta usando `printf`, una línea por `printf` (pista: `%-14s` rellena un texto hasta 14 caracteres, y `%4d` escribe un entero en 4 espacios). El total lo sumas tú:

Completo	2500
Bebida	1200
TOTAL	3700

Está desarrollado, con una tercera línea y la de guiones, en el **B2.2**.

✓ **Al terminar el paso 2**, tu código tiene que hacer lo mismo que `Paso02Imprimir.java`.

Compruébalo con `javac Paso02Imprimir.java` y `java Paso02Imprimir`, y si algo no calza, compara los dos archivos línea por línea.

 **Ejercicios del paso 2:** [paso02/ejercicios.md](#)

Paso 3 - Variables y operaciones

Objetivo: guardar datos en variables con tipo, operar con ellos y conocer las dos trampas de la división.

Crea `Paso03Variables.java` con el mismo esqueleto del paso 2 (la clase y el `main` vacío). Todo lo que sigue va **dentro del `main`**, un bloque debajo del otro.

3.1 Declarar: tipo, nombre y valor

```
int edad = 20;
double estatura = 1.75;
boolean esAlumno = true;
char seccion = 'A';
String nombre = "Ana";

System.out.println(nombre + " tiene " + edad + " años y mide " + estatura);
System.out.println("Seccion " + seccion + ", es alumno: " + esAlumno);
```

```
Ana tiene 20 años y mide 1.75
Seccion A, es alumno: true
```

Tipo	Guarda	Ejemplo
<code>int</code>	enteros	<code>20</code> , <code>-3</code>

Tipo	Guarda	Ejemplo
<code>double</code>	decimales	<code>1.75</code> (con punto)
<code>boolean</code>	verdadero o falso	<code>true</code> , <code>false</code>
<code>char</code>	un carácter, comillas simples	<code>'A'</code>
<code>String</code>	texto, comillas dobles	<code>"Ana"</code>

Fíjate que `String` es el único que va con **mayúscula**. No es un error de tipeo: `String` es una **clase**, y las clases en Java empiezan con mayúscula. Los otros cuatro son tipos *primitivos*. En la demo 1 vas a crear tus propias clases, y se escriben igual.

3.2 Cambiar el valor

```
edad = edad + 1;           // 21
edad++;                   // 22, sumar 1
edad += 5;                // 27, sumar 5
System.out.println("Edad ahora: " + edad);
```

Edad ahora: 27

El tipo se escribe **solo la primera vez**. Después, la variable ya existe y solo se le asigna un valor nuevo.

3.3 Operaciones

```
System.out.println(7 + 2);    // 9
System.out.println(7 * 2);    // 14
System.out.println(7 / 2);    // 3 !!
System.out.println(7 % 2);    // 1, el resto de la division
System.out.println(7 / 2.0);  // 3.5
```

Primera trampa: `7 / 2` da **3**. Entero dividido por entero da **entero**: el decimal se bota, no se redondea. Basta con que uno de los dos sea decimal (`2.0`) para que el resultado lo sea.

El `%` (módulo) es el **resto** de la división. Sirve más de lo que parece: `n % 2 == 0` es la forma de preguntar si `n` es par.

3.4 Convertir de tipo (cast)

```
int suma = 17;
int cantidad = 3;
double malo = suma / cantidad;
double bueno = (double) suma / cantidad;
int sinDecimales = (int) 5.99;
System.out.println(malo + " / " + bueno + " / " + sinDecimales);
```

```
5.0 / 5.666666666666667 / 5
```

Segunda trampa: guardar en un `double` no arregla nada. En `malo`, Java primero divide dos enteros (da 5) y **después** lo guarda como 5.0. Hay que convertir **antes** de dividir: eso hace `(double) suma`.

Y `(int) 5.99` da 5: el cast corta, no redondea.

3.5 Constantes

```
final double NOTA_MINIMA = 4.0;
System.out.println("Se aprueba con " + NOTA_MINIMA);
```

`final` significa que no se puede volver a asignar. Por costumbre, las constantes van en `MAYUSCULAS_CON_GUION_BAJO`.

3.6 `Math`: lo que no tiene símbolo

```
System.out.println(Math.sqrt(16));           // 4.0
System.out.println(Math.pow(2, 10));          // 1024.0
System.out.println(Math.round(5.5));          // 6
System.out.println(Math.max(3, 8));           // 8
```

Java **no** tiene operador de potencia. `2 ^ 10` compila, pero da 8: el `^` es otra cosa. Para potencias se usa `Math.pow`.

Rómpelo

Cambio	Lo que dice <code>javac</code>
<code>int edad = "veinte";</code>	<code>incompatible types: String cannot be converted to int</code>

Cambio	Lo que dice <code>javac</code>
<code>string nombre</code> con minúscula	<code>cannot find symbol ... class string</code>
Cambia <code>int edad = 20;</code> por <code>int edad;</code> , sin tocar el resto	<code>variable edad might not have been initialized</code>
Agrega <code>NOTA_MINIMA = 3.5;</code> después de declararla	<code>cannot assign a value to final variable NOTA_MINIMA</code>

Si vienes de Python

En Python `edad = 20` crea la variable, y después puedes hacer `edad = "veinte"` sin problema. En Java la variable **nace con un tipo y muere con él**. El primer error del cuadro de arriba no existe en Python.

Python	Java
<code>7 // 2 → 3</code>	<code>7 / 2 → 3</code> (entre enteros)
<code>7 / 2 → 3.5</code>	<code>7 / 2.0 → 3.5</code>
<code>2 ** 10</code>	<code>Math.pow(2, 10)</code>
<code>x += 1</code>	<code>x += 1</code> o <code>x++</code>
<code>True / False</code>	<code>true / false</code> (minúscula)
<code>int(5.99)</code>	<code>(int) 5.99</code>

Ahora tú

- Un precio neto de 10000. Declara la constante `IVA = 0.19` e imprime el IVA y el total.
- Una película dura 135 minutos. Usa `/` y `%` para imprimir `Dura 2 horas y 15 minutos`.

✓ **Al terminar el paso 3**, tu código tiene que hacer lo mismo que `Paso03Variables.java`.

Compruébalo con `javac Paso03Variables.java` y `java Paso03Variables`, y si algo no calza, compara los dos archivos línea por línea.

 **Ejercicios del paso 3:** [paso03/ejercicios.md](#)

Paso 4 - Leer del teclado con `Scanner`

Objetivo: pedirle datos al usuario. Y conocer las dos trampas de `Scanner`, porque te van a pasar sí o sí.

Crea `Paso04Teclado.java`. Esta vez hay una línea **antes** de la clase:

```
import java.util.Scanner;

public class Paso04Teclado {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

    }
}
```

`Scanner` no viene incluido: hay que **importarlo**. Y la línea `Scanner sc = new Scanner(System.in);` crea un lector conectado al teclado (`System.in`). Se crea **uno solo**, al principio, y se usa todo el programa.

4.1 Leer un texto

```
System.out.print("Como te llamas? ");
String nombre = sc.nextLine();

System.out.println("Hola " + nombre);
```

Fíjate que la pregunta usa `print` y no `println`: así el cursor queda en la misma línea de la pregunta.

```
Como te llamas? Ana Perez
Hola Ana Perez
```

4.2 Leer números

Borra el `System.out.println("Hola " + nombre);` y agrega:

```
System.out.print("Cuantos años tienes? ");
int edad = sc.nextInt();

System.out.print("Cuanto mides en metros? ");
double estatura = sc.nextDouble();

System.out.println("El próximo año vas a tener " + (edad + 1) + " años");
System.out.printf("Mides %.2f metros\n", estatura);
```

Para leer	Se usa
una línea de texto	<code>sc.nextLine()</code>
un entero	<code>sc.nextInt()</code>
un decimal	<code>sc.nextDouble()</code>

Ejecútalo y escribe `1.68` en la estatura.

Primera trampa: la coma decimal. Si el programa se cae con `InputMismatchException` al escribir `1.68`, tu PC está configurado en español, y en español los decimales se escriben con **coma**. Prueba con `1,68`. Y fíjate que el `printf` también imprime con coma.

Esto solo le pasa a `Scanner` y a `printf`, que respetan el idioma del PC. **En el código** los decimales se escriben siempre con punto: `1.68`. Si prefieres que el programa lea con punto en cualquier PC, crea el `Scanner` así: `new Scanner(System.in).useLocale(java.util.Locale.US)`.

4.3 La trampa del Enter

Ahora agrega una pregunta de texto **después** de los números. Pon este bloque justo después del `nextDouble`:

```
System.out.print("Que carrera estudias? ");
String carrera = sc.nextLine();
System.out.println("Estudias " + carrera);
```

Ejecútalo. **El programa no te deja escribir la carrera:** la salta y termina.

```
Cuanto mides en metros? 1.68
Que carrera estudias? Estudias
```

Qué pasó. Cuando escribiste `1.68` y apretaste Enter, llegó `1.68` + Enter. `nextDouble()` se llevó el número y **dejó el Enter**. Después, `nextLine()` lee "hasta el próximo Enter"... que ya estaba ahí. Leyó una línea vacía.

El arreglo: después de un `nextInt()` o `nextDouble()`, si lo que sigue es un `nextLine()`, hay que comerse el Enter con un `nextLine()` suelto:

```
double estatura = sc.nextDouble();
sc.nextLine();           // se come el Enter que dejo nextDouble
```

4.4 Ordena y cierra

Tu programa quedó con las preguntas y las respuestas mezcladas. Déjalo en tres partes, en este orden:

1. **Las cuatro preguntas**, una debajo de la otra: nombre, edad, estatura (con su `sc.nextLine()` de más) y carrera. Ya las tienes escritas; si alguna quedó más abajo, súbela.
2. **Borra las tres líneas que imprimen resultados** (la del próximo año, la de `Mides` y la de `Estudias`) y escribe **al final**, todas juntas, estas cuatro:

```
System.out.println();
System.out.println("Hola " + nombre + ", de " + carrera);
System.out.println("El proximo anio vas a tener " + (edad + 1) + " anos");
System.out.printf("Mides %.2f metros\n", estatura);
```

3. `sc.close();` como última línea del `main`: cierra el Scanner cuando ya no se va a leer nada más.

Deberías ver (lo que va después de cada pregunta lo escribes tú):

```
Como te llamas? Ana Perez
Cuantos anios tienes? 20
Cuanto mides en metros? 1.68
Que carrera estudias? Informatica

Hola Ana Perez, de Informatica
El proximo anio vas a tener 21 anos
Mides 1.68 metros
```

Rómpelo

Cambio	Qué pasa
Borra el <code>import</code>	<code>cannot find symbol ... class Scanner</code> (dos veces: una por cada <code>Scanner</code> de la línea)
Ejecuta y escribe <code>veinte</code> en la edad	<code>Exception in thread "main"</code> <code>java.util.InputMismatchException</code>

El segundo **compila** y se cae al ejecutar. Es tu primera excepción: la demo 10 entera trata de cómo evitar que el programa se muera así.

Si vienes de Python

```
nombre = input("Como te llamas? ")
edad = int(input("Cuantos años tienes? "))
```

En Python `input()` **siempre** devuelve texto y tú lo conviertes. En Java le pides a `Scanner` el tipo que quieres leer. La trampa del Enter **no existe** en Python, porque `input()` siempre se lleva la línea completa.

Ahora tú

- Pide dos enteros e imprime su suma, su resta, su multiplicación y su división **con decimales** (acuérdate del paso 3).
- Pide el precio neto de un producto y la cantidad. Imprime el total con IVA, con `printf` y sin decimales (`%.0f`).

✓ **Al terminar el paso 4**, tu código tiene que hacer lo mismo que [Paso04Teclado.java](#).

Compruébalo con `javac Paso04Teclado.java` y `java Paso04Teclado`, y si algo no calza, compara los dos archivos línea por línea.

 **Ejercicios del paso 4:** [paso04/ejercicios.md](#)

Paso 5 - Decidir: `if`, `else`, `switch`

Objetivo: que el programa haga cosas distintas según los datos.

Crea `Paso05Decidir.java` con el mismo esqueleto del paso 4: el `import java.util.Scanner;` arriba de la clase, el `main`, y adentro la línea `Scanner sc = new Scanner(System.in);`. Los bloques que siguen van debajo de esa línea.

5.1 if / else if / else

```
System.out.print("Ingresa tu nota (1.0 a 7.0): ");
double nota = sc.nextDouble();
sc.nextLine();

if (nota < 1.0 || nota > 7.0) {
    System.out.println("Esa nota no existe");
} else if (nota >= 4.0) {
    System.out.println("Aprobado");
} else {
    System.out.println("Reprobado");
}
```

Pruébalo con `5.5`, con `3.2` y con `9`.

Se revisan **en orden** y se ejecuta **solo el primero** que se cumple. Por eso el `9` no dice "Aprobado", aunque sea mayor que 4.0: nunca llega a esa pregunta.

Compara	Combina
<code>==</code> igual	<code>&&</code> y (las dos)
<code>!=</code> distinto	<code> </code> o (al menos una)
<code><</code> <code><=</code> <code>></code> <code>>=</code>	<code>!</code> no

La condición va **siempre entre paréntesis**, y el bloque **entre llaves**. Java permite omitir las llaves cuando el bloque tiene una sola línea, pero no lo hagas: es la fuente de errores más tonta que existe.

5.2 Comparar textos: `equals`, nunca `==`

```
System.out.print("Clave: ");
String clave = sc.nextLine();

if (clave.equals("duoc")) {
    System.out.println("Bienvenido");
} else {
    System.out.println("Clave incorrecta");
}
```

Ahora cámbialo por `if (clave == "duoc")`, compila, ejecuta y escribe `duoc`:

Clave: duoc
Clave incorrecta

Compila sin un solo aviso y está mal. Con textos, `==` no pregunta si tienen las mismas letras: pregunta si son **el mismo objeto** en la memoria. El texto que escribiste en el teclado es un objeto nuevo, así que nunca es "el mismo" que `"duoc"`.

Para textos, **siempre** `equals`. (Y `equalsIgnoreCase` si no te importan las mayúsculas.) Vuelve a dejarlo con `equals`.

Esto se entiende del todo en la demo 6, que trata justamente de `==` contra `equals()`. Por ahora, la regla: números con `==`, textos con `equals`.

5.3 `switch`: elegir entre valores exactos

```
System.out.print("Dia de la semana (1 a 7): ");
int dia = sc.nextInt();

switch (dia) {
    case 1: System.out.println("Lunes"); break;
    case 2: System.out.println("Martes"); break;
    case 3: System.out.println("Miercoles"); break;
    case 4: System.out.println("Jueves"); break;
    case 5: System.out.println("Viernes"); break;
    case 6:
    case 7: System.out.println("Fin de semana"); break;
    default: System.out.println("Ese dia no existe");
}

sc.close();
```

- Cada `case` termina con `break`. Si lo olvidas, **sigue de largo** y ejecuta también el siguiente.
- El 6 y el 7 comparten el mismo código justamente por eso: el 6 no tiene `break` y cae en el 7.
- `default` es el `else` del `switch`.

Usa `switch` cuando comparas **una** variable con valores **exactos** (1, 2, 3...). Para rangos (nota `>= 4.0`) sirve solo el `if`.

Rómpelo

Cambio	Qué pasa
<code>else if (nota = 4.0)</code> , con un solo <code>=</code>	<code>incompatible types: double cannot be converted to boolean</code>
Borra el <code>break</code> del <code>case 1</code> y escribe <code>1</code>	Imprime <code>Lunes</code> y <code>Martes</code>

El primero es un regalo del compilador: un `=` es *asignar*, dos `==` es *comparar*. Java no te deja confundirlos en un `if`.

Si vienes de Python

Python	Java
<code>if x > 3:</code>	<code>if (x > 3) {</code>
<code>elif</code>	<code>else if</code>
<code>and</code> <code>or</code> <code>not</code>	<code>&&</code> <code> </code> <code>!</code>
la sangría marca el bloque	las llaves marcan el bloque (la sangría es solo para leer)
<code>nombre == "duoc"</code> compara letras	<code>nombre.equals("duoc")</code> compara letras
<code>match</code> (3.10+)	<code>switch</code>

La del `==` es la diferencia más traicionera de todo el paso: en Python `==` con textos funciona perfecto, y en Java compila y falla calladito.

Ahora tú

- Pide un número y di si es par o impar (pista: `%`, paso 3).
- Pide la edad y di si es menor de edad (menos de 18), adulto (18 a 64) o adulto mayor (65 o más).
- Haz que la clave funcione también si escriben `DUOC` o `Duoc`.

✓ **Al terminar el paso 5**, tu código tiene que hacer lo mismo que [Paso05Decidir.java](#).

Compruébalo con `javac Paso05Decidir.java` y `java Paso05Decidir`, y si algo no calza, compara los dos archivos línea por línea.

 **Ejercicios del paso 5:** [paso05/ejercicios.md](#)

Paso 6 - Repetir con `while`

Objetivo: repetir mientras algo se cumpla. Validar datos y armar un menú.

Crea `Paso06While.java` con el esqueleto del paso 4 (el `import`, el `main` y el `Scanner sc = new Scanner(System.in);`).

6.1 Contar

```
int i = 1;
while (i <= 5) {
    System.out.println("Vuelta " + i);
    i++;
}
```

```
Vuelta 1
Vuelta 2
Vuelta 3
Vuelta 4
Vuelta 5
```

Mientras la condición sea verdadera, se ejecuta el bloque y **se vuelve a preguntar**. Algo dentro del bloque tiene que cambiar la condición (aquí, el `i++`), o no termina nunca.

Pruébalo: borra el `i++` y ejecuta. `Vuelta 1` sin parar. Se corta con **Ctrl + C**. Todos los que programan han hecho esto alguna vez. **Vuelve a poner el `i++`** antes de seguir con el 6.2.

6.2 Validar: pedir otra vez mientras el dato esté malo

```
System.out.print("Ingresa una nota (1.0 a 7.0): ");
double nota = sc.nextDouble();
while (nota < 1.0 || nota > 7.0) {
    System.out.print("Nota invalida. Otra vez: ");
    nota = sc.nextDouble();
}
System.out.println("Nota aceptada: " + nota);
```

```
Ingresa una nota (1.0 a 7.0): 9
Nota invalida. Otra vez: 0.5
Nota invalida. Otra vez: 5.5
Nota aceptada: 5.5
```

Aquí se ve para qué sirve de verdad el `while`: **no sabes cuántas vueltas va a dar**. Depende de cuántas veces se equivoque el usuario.

Compáralo con el `if` del paso 5: el `if` pregunta **una vez**; el `while` pregunta **hasta que** la respuesta sea buena.

6.3 `do-while`: un menú

```
int opcion;
do {
    System.out.println();
    System.out.println("1) Saludar");
    System.out.println("2) Contar hasta 3");
    System.out.println("0) Salir");
    System.out.print("Opcion: ");
    opcion = sc.nextInt();

    switch (opcion) {
        case 1: System.out.println("Hola!"); break;
        case 2: System.out.println("1, 2, 3"); break;
        case 0: System.out.println("Chao"); break;
        default: System.out.println("Esa opcion no existe");
    }
} while (opcion != 0);

sc.close();
```

El `do-while` pregunta **al final**, así que el bloque corre al menos **una** vez. Es exactamente lo que necesita un menú: primero se muestra, después se decide si se repite.

Fíjate que `int opcion;` se declara **antes** del `do`. Si la declaras adentro, el `while (opcion != 0)` del final no la ve: una variable solo existe dentro de las llaves donde nació.

Rómpelo

Cambio	Qué pasa
Mueve <code>int opcion;</code> dentro del <code>do</code> (como <code>int opcion = sc.nextInt();</code>)	<code>cannot find symbol ... variable opcion</code> , en la línea del <code>while</code>
En el menú, escribe <code>abc</code>	<code>InputMismatchException</code> . Un menú que se muere con una letra: eso lo arregla la demo 10.

Si vienes de Python

El `while` es igual, con paréntesis y llaves. El que no existe en Python es el `do-while`: allá se simula con un `while True:` y un `break`.

```
while True:
    opcion = int(input("Opcion: "))
    if opcion == 0:
        break
```

Ahora tú

- Pide números hasta que el usuario escriba `0`, y al final muestra la suma de todos.
- **Adivina el número.** Declara `int secreto = 7;`. Pide números hasta que acierte, diciendo "mas alto" o "mas bajo" en cada intento. Al final, cuántos intentos le tomó. (Extra: que el secreto sea al azar con `(int) (Math.random() * 100) + 1`.)

✓ **Al terminar el paso 6**, tu código tiene que hacer lo mismo que [Paso06While.java](#). Compruébalo con `javac Paso06While.java` y `java Paso06While`, y si algo no calza, compara los dos archivos línea por línea.

📄 **Ejercicios del paso 6:** [paso06/ejercicios.md](#)

Paso 7 - Repetir con `for`

Objetivo: repetir un número conocido de veces, acumular, y anidar repeticiones.

Crea `Paso07For.java` con el esqueleto del paso 4 (el `import`, el `main` y el `Scanner sc = new Scanner(System.in);`).

7.1 El mismo conteo, en una línea

```
for (int i = 1; i <= 5; i++) {  
    System.out.println("Vuelta " + i);  
}
```

Imprime lo mismo que el 6.1. El `for` junta en **una línea** las tres cosas que en el `while` estaban repartidas:

```
for ( int i = 1 ;    i <= 5 ;    i++ )  
    donde parte    hasta cuando    como avanza
```

¿ `for` o `while` ? Si sabes cuántas vueltas son (5 alumnos, 10 filas, el largo de un arreglo): `for`. Si depende de algo que no sabes (el usuario escribe mal, elige salir): `while`.

7.2 Hacia atrás y de a dos

```
for (int i = 10; i >= 0; i -= 2) {  
    System.out.print(i + " ");  
}  
System.out.println();
```

10 8 6 4 2 0

7.3 Acumular

```
int suma = 0;  
for (int i = 1; i <= 100; i++) {  
    suma += i;  
}  
System.out.println("Suma del 1 al 100: " + suma);
```

Suma del 1 al 100: 5050

Es el patrón **acumulador**: una variable que nace en 0 **antes** del `for`, y a la que se le va sumando algo en cada vuelta. Lo vas a usar para promedios en los pasos 8, 9, 10 y 13.

7.4 Una tabla de multiplicar

```
System.out.print("Que tabla quieres? ");
int tabla = sc.nextInt();
for (int i = 1; i <= 10; i++) {
    System.out.printf("%d x %2d = %3d%n", tabla, i, tabla * i);
}
```

```
Que tabla quieres? 7
7 x  1 =   7
7 x  2 =  14
...
7 x 10 =  70
```

El `%2d` y el `%3d` reservan 2 y 3 espacios: por eso la tabla sale alineada.

7.5 Un `for` dentro de otro

```
for (int fila = 1; fila <= 4; fila++) {
    for (int col = 1; col <= fila; col++) {
        System.out.print("*");
    }
    System.out.println();
}
```

```
*
**
***
****
```

El de afuera recorre las **filas**; por cada fila, el de adentro dibuja tantas estrellas como el número de la fila. El `println()` va **entre** los dos `}`: al terminar cada fila.

7.6 `continue` y `break`

```
for (int i = 1; i <= 10; i++) {  
    if (i % 2 == 0) {  
        continue;  
    }  
    if (i > 7) {  
        break;  
    }  
    System.out.print(i + " ");  
}  
System.out.println();  
  
sc.close();
```

1 3 5 7

`continue` salta **a la siguiente vuelta** (se salta los pares). `break` **corta el `for` entero** (cuando pasa de 7, se acabó).

Rómpelo

Cambio	Qué pasa
Después del <code>for</code> del 7.3, agrega <code>System.out.println(i);</code>	<code>cannot find symbol ... variable i</code> : la <code>i</code> nació dentro del <code>for</code> y murió con él
En el 7.5, mueve el <code>println()</code> dentro del <code>for</code> de adentro	Una estrella por línea: 10 líneas en vez de 4

Si vienes de Python

Python	Java
<code>for i in range(5):</code>	<code>for (int i = 0; i < 5; i++) {</code>
<code>for i in range(1, 6):</code>	<code>for (int i = 1; i <= 5; i++) {</code>
<code>for i in range(10, -1, -2):</code>	<code>for (int i = 10; i >= 0; i -= 2) {</code>

El `for` de Java es más largo, pero no esconde nada: el inicio, el final y el paso están escritos a la vista. `continue` y `break` funcionan igual.

Ahora tú

- Cuenta cuántos múltiplos de 3 hay entre 1 y 100.
- Calcula el factorial de un número que pida el usuario ($5! = 1*2*3*4*5 = 120$).
- Dibuja el triángulo al revés (4, 3, 2, 1 estrellas).
- Imprime las tablas del 1 al 10 completas, con un `for` dentro de otro.

✓ **Al terminar el paso 7**, tu código tiene que hacer lo mismo que `Paso07For.java`. Compruébalo con `javac Paso07For.java` y `java Paso07For`, y si algo no calza, compara los dos archivos línea por línea.

 **Ejercicios del paso 7:** [paso07/ejercicios.md](#)

Paso 8 - Arreglos

Objetivo: guardar muchos valores en una sola variable y recorrerlos.

Crea `Paso08Arreglos.java`. Esta vez **sin** Scanner, pero con otro import, que va igual que siempre **arriba de la clase**:

```
import java.util.Arrays;
```

8.1 Crear un arreglo

```
double[] notas = {5.5, 6.2, 3.8, 7.0};

System.out.println("Cantidad de notas: " + notas.length);
System.out.println("Primera: " + notas[0]);
System.out.println("Ultima: " + notas[notas.length - 1]);
```

```
Cantidad de notas: 4
Primera: 5.5
Ultima: 7.0
```

Un arreglo es una fila de **casilleros del mismo tipo**, numerados **desde 0**:

notas	[	5.5		6.2		3.8		7.0	]
indice		0		1		2		3	length = 4

Como parte en 0, el último es `length - 1`. Y `length` va **sin paréntesis** (al revés que `nombre.length()` en un `String`: sí, es inconsistente, y sí, te vas a equivocar).

8.2 Cambiar un casillero e imprimir el arreglo

```
notas[2] = 4.5;

System.out.println(notas);
System.out.println(Arrays.toString(notas));
```

```
[D@659e0bfd
[5.5, 6.2, 4.5, 7.0]
```

¿Qué es `[D@659e0bfd` ? Es lo que Java sabe decir de un arreglo si no le explicas cómo imprimirlo: "un arreglo de *Doubles*, que vive en tal lugar de la memoria". Para ver el contenido se usa `Arrays.toString` . (El número cambia en cada ejecución.)

Vas a volver a ver esta basura en la demo 6, con tus propios objetos. Ahí aprendes a arreglarla con `toString()` .

8.3 Recorrer con `for`

```
for (int i = 0; i < notas.length; i++) {
    System.out.println("Nota " + (i + 1) + ": " + notas[i]);
}
```

```
Nota 1: 5.5
Nota 2: 6.2
Nota 3: 4.5
Nota 4: 7.0
```

Mira la condición: `i < notas.length` , con `<` y **no** `<=` . El último índice es 3, no 4.

8.4 Recorrer con *for-each*

```
double suma = 0;
double mayor = notas[0];
for (double n : notas) {
    suma += n;
    if (n > mayor) {
        mayor = n;
    }
}
System.out.printf("Promedio: %.1f, mayor: %.1f%n", suma / notas.length,
mayor);
```

```
Promedio: 5.8, mayor: 7.0
```

`for (double n : notas)` se lee "**para cada** `n` **en** `notas`". Es más corto, pero no tienes la posición. Úsalo cuando solo te importa el valor.

El truco del mayor: parte suponiendo que el mayor es **el primero**, y en cada vuelta pregunta si el actual le gana.

8.5 Un arreglo vacío

```
String[] nombres = new String[3];
nombres[0] = "Ana";
nombres[1] = "Luis";
System.out.println(Arrays.toString(nombres));
```

```
[Ana, Luis, null]
```

Con `new` se da el **largo**, y los valores se ponen después. El que nadie llenó quedó en `null`: "aquí no hay nada". (Un `int[]` vacío viene con ceros.)

8.6 Un adelanto: `ArrayList`

El arreglo tiene un problema: su largo es **fijo**. Si nace con 3 casilleros, muere con 3. Para una lista que crece se usa `ArrayList`. Agrega el import `import java.util.ArrayList;` y al final del `main`:

```

ArrayList<String> ramos = new ArrayList<>();
ramos.add("P00");
ramos.add("Base de datos");
ramos.add("Ingles");
System.out.println(ramos.size() + " ramos: " + ramos);
System.out.println("El segundo es " + ramos.get(1));

```

3 ramos: [P00, Base de datos, Ingles]
 El segundo es Base de datos

	Arreglo	ArrayList
Crear	<code>new String[3]</code>	<code>new ArrayList<>()</code>
Largo	fijo	crece con <code>add</code>
Leer	<code>nombres[1]</code>	<code>ramos.get(1)</code>
Cuántos	<code>nombres.length</code>	<code>ramos.size()</code>
Imprimir	<code>Arrays.toString(nombres)</code>	directo

Lo vas a usar en serio desde la demo 5. Por ahora basta con saber que existe.

Rómpelo

Cambio	Qué pasa
Imprime <code>notas[4]</code>	Compila, y al ejecutar: <code>ArrayIndexOutOfBoundsException: Index 4 out of bounds for length 4</code>
En el 8.3, cambia <code><</code> por <code><=</code>	Imprime las 4 notas... y después se cae con la misma excepción

Si vienes de Python

Cuidado: la *lista* de Python **no** es el arreglo de Java. Se parece mucho más a `ArrayList`.

Python	Java (arreglo)	Java (ArrayList)
<code>notas = [5.5, 6.2]</code>	<code>double[] notas = {5.5, 6.2};</code>	
<code>len(notas)</code>	<code>notas.length</code>	<code>ramos.size()</code>

Python	Java (arreglo)	Java (ArrayList)
<code>notas.append(4.0)</code>	no se puede	<code>ramos.add(...)</code>
<code>for n in notas:</code>	<code>for (double n : notas) {</code>	igual
<code>print(notas)</code>	<code>Arrays.toString(notas)</code>	directo
<code>notas[-1]</code>	<code>notas[notas.length - 1]</code>	

Y una lista de Python puede mezclar tipos (`[1, "hola", True]`). Un arreglo de Java, no: un `double[]` guarda solo `double` .

Ahora tú

- Pide 5 notas por teclado y guárdalas en un `double[] notas = new double[5];` (paso 4 + paso 7 + paso 8).
- Busca la nota **menor**.
- Cuenta cuántas notas son rojas (menores a 4.0).

✓ **Al terminar el paso 8**, tu código tiene que hacer lo mismo que `Paso08Arreglos.java` .

Compruébalo con `javac Paso08Arreglos.java` y `java Paso08Arreglos` , y si algo no calza, compara los dos archivos línea por línea.

 **Ejercicios del paso 8:** [paso08/ejercicios.md](#)

Paso 9 - Métodos

Objetivo: sacar código del `main` y ponerle nombre. Y, de paso, entender por fin la primera línea del `main` .

Crea `Paso09Metodos.java` . Los métodos van **dentro de la clase** pero **fuera del `main`** : entre las llaves de la clase, antes o después del `main` .

```
public class Paso09Metodos {

    // aqui van los metodos

    public static void main(String[] args) {

    }

}
```

9.1 Un método que hace algo: `void`

Arriba del `main`:

```
static void linea() {  
    System.out.println("-----");  
}
```

Y dentro del `main`, llámalo dos veces:

```
linea();  
linea();
```

`void` significa que el método **no devuelve nada**: hace algo y listo.

9.2 Con un parámetro

Este segundo método va **debajo de `linea()`**, también fuera del `main`:

```
static void saludar(String nombre) {  
    System.out.println("Hola, " + nombre);  
}
```

Y en el `main`, reemplaza las dos llamadas del 9.1 por estas cuatro líneas:

```
linea();  
saludar("Ana");  
saludar("Luis");  
linea();
```

```
-----  
Hola, Ana  
Hola, Luis  
-----
```

El dato entra por los paréntesis. Dentro del método, `nombre` es una variable normal que vale lo que le pasaron.

9.3 Un método que responde algo: `return`

Otros dos métodos, debajo de los anteriores y fuera del `main`:

```

static double promedio(double[] notas) {
    double suma = 0;
    for (double n : notas) {
        suma += n;
    }
    return suma / notas.length;
}

static boolean aprobo(double promedio) {
    return promedio >= 4.0;
}

```

La palabra de la izquierda (`double`, `boolean`) es **lo que devuelve**. Y el `return` es lo que responde. Agrega al final del `main`:

```

double[] notasAna = {5.5, 6.2, 4.5};
double[] notasLuis = {3.2, 4.1, 3.5};

double pAna = promedio(notasAna);
double pLuis = promedio(notasLuis);

System.out.printf("Ana: %.1f aprobo: %b\n", pAna, aprobo(pAna));
System.out.printf("Luis: %.1f aprobo: %b\n", pLuis, aprobo(pLuis));
linea();

```

```

Ana: 5.4 aprobo: true
Luis: 3.6 aprobo: false

```

Mira lo que ganaste: el cálculo del promedio está escrito **una vez** y se usa **dos**. Si mañana hay que cambiarlo, se cambia en un solo lugar.

9.4 Ahora sí: la primera línea del `main`

```
public static void main(String[] args)
```

Palabra	Ya sabes que significa
<code>public</code>	se puede usar desde fuera de la clase (lo ve <code>java</code> para arrancar)
<code>static</code>	lo mismo que tus métodos de este paso (abajo)

Palabra	Ya sabes que significa
<code>void</code>	no devuelve nada (9.1)
<code>main</code>	el nombre. <code>java</code> busca uno que se llame exactamente así
<code>String[] args</code>	un parámetro (9.2) que es un arreglo de textos (paso 8)

El `main` es un método, igual que `linea()`. Lo único especial es que a ese lo llama Java, no tú.

¿Y el `static`? Por ahora todos los métodos lo llevan porque todavía no hay **objetos**: el método le pertenece a la clase, y se llama directo. Desde la demo 1 vas a escribir métodos **sin** `static`, que le pertenecen a cada objeto. Ese es el salto a la programación orientada a objetos.

Rómpelo

Cambio	Lo que dice <code>javac</code>
Borra el <code>return</code> de <code>promedio</code>	<code>missing return statement</code>
Guarda el promedio en un <code>int</code> : <code>int x = promedio(notasAna);</code>	<code>possible lossy conversion from double to int</code>
Borra el <code>static</code> de <code>linea()</code>	<code>non-static method linea() cannot be referenced from a static context</code>

Guárdate el último. Es el primer error que le aparece a todo el mundo cuando empieza con objetos, y su explicación completa es la demo 1.

Si vienes de Python

Python	Java
<code>def promedio(notas):</code>	<code>static double promedio(double[] notas) {</code>
el parámetro no tiene tipo	cada parámetro lleva su tipo
no se declara qué devuelve	el tipo de retorno va antes del nombre (<code>void</code> si nada)
una función sin <code>return</code> devuelve <code>None</code>	un método no <code>void</code> sin <code>return</code> no compila

Ahora tú

- Escribe `static boolean esPar(int n)` y úsalo en un `for` del 1 al 10.
- Escribe `static double maximo(double[] notas)`.

- Convierte la tabla de multiplicar del paso 7 en `static void tabla(int n)`, y llámala para las tablas del 3, del 7 y del 9.

✓ **Al terminar el paso 9**, tu código tiene que hacer lo mismo que `Paso09Metodos.java`.

Compruébalo con `javac Paso09Metodos.java` y `java Paso09Metodos`, y si algo no calza, compara los dos archivos línea por línea.

 **Ejercicios del paso 9:** [paso09/ejercicios.md](#)

Paso 10 - Clases con métodos `static`

Objetivo: mudar los métodos a una clase propia, en su propio archivo, y usarlos desde otra. Es exactamente lo que hace `Math` desde el paso 3.

10.1 Una clase sin `main`

Crea un archivo nuevo, `Notas.java`:

```
public class Notas {

    public static final double NOTA_MINIMA = 1.0;
    public static final double NOTA_MAXIMA = 7.0;
    public static final double NOTA_APROBACION = 4.0;

    public static double promedio(double[] notas) {
        double suma = 0;
        for (double n : notas) {
            suma += n;
        }
        return suma / notas.length;
    }

    public static boolean aprobo(double promedio) {
        return promedio >= NOTA_APROBACION;
    }
}
```

Tres cosas nuevas:

- **No tiene `main`**. Nadie la ejecuta: otras clases la usan. Es una caja de herramientas.
- **Los métodos llevan `public`**, porque ahora los va a llamar **otra** clase. (`public` y `private` se ven en serio en la demo 2.)

- Las constantes viven en la clase, fuera de cualquier método, y también son `public static`.

10.2 Usarla desde otra clase

Crea `Paso10Clases.java`, en la **misma carpeta**:

```
public class Paso10Clases {  
    public static void main(String[] args) {  
        double[] notas = {5.5, 6.2, 3.8, 4.5};  
  
        double p = Notas.promedio(notas);  
        System.out.printf("Promedio: %.1f%n", p);  
        System.out.println("Aprobo: " + Notas.aprobo(p));  
        System.out.println("Se aprueba con " + Notas.NOTA_APROBACION);  
    }  
}
```

Ahora son **dos archivos**, y se compilan los dos:

```
javac Notas.java Paso10Clases.java  
java Paso10Clases
```

```
Promedio: 5.0  
Aprobo: true  
Se aprueba con 4.0
```

Haz `ls`: hay un `.class` por cada clase. Y se ejecuta la que tiene `main`.

La regla para llamar a un método:

- Si está en **tu misma** clase, se llama por su nombre: `promedio(notas)` (así lo hiciste en el paso 9).
- Si está en **otra** clase, va el nombre de la clase adelante: `Notas.promedio(notas)`.

Fíjate en `Notas.promedio(notas)`: `Notas` con mayúscula es **la clase**; `notas` con minúscula es **tu arreglo**. Para Java son dos cosas que no tienen nada que ver.

Atajo: `javac Paso10Clases.java` solo también funciona: `javac` ve que usas `Notas` y compila `Notas.java` por su cuenta. Las demos del curso usan `javac *.java`, que compila **todo** lo de la carpeta. Ojo: también tus pasos anteriores, así que si dejaste alguno roto de un "Rómpelo", falla.

10.3 Crecer la caja de herramientas

Agrega estos dos métodos a `Notas.java`:

```
public static double maxima(double[] notas) {
    double mayor = notas[0];
    for (double n : notas) {
        if (n > mayor) {
            mayor = n;
        }
    }
    return mayor;
}

public static int contarRojas(double[] notas) {
    int rojas = 0;
    for (double n : notas) {
        if (n < NOTA_APROBACION) { // dentro de la clase, sin "Notas."
            rojas++;
        }
    }
    return rojas;
}
```

Y úsalos en el `main`, debajo del promedio:

```
System.out.println("Nota maxima: " + Notas.maxima(notas));
System.out.println("Notas rojas: " + Notas.contarRojas(notas));
```

Compila **los dos archivos** otra vez: cambiaste `Notas.java`.

10.4 Ya conocías una clase así

Agrega al final del `main`:

```
System.out.println("Raiz de 16: " + Math.sqrt(16));
System.out.println("PI: " + Math.PI);
```

`Math` es exactamente lo que acabas de escribir: una clase llena de métodos `static` y constantes.

`Math.sqrt` es tu `Notas.promedio`; `Math.PI` es tu `Notas.NOTA_APROBACION`. La diferencia es que

`Math` la escribió otra persona.

La versión terminada: `paso10/referencia/Notas.java` y `paso10/referencia/Paso10Clases.java`.

Una pregunta para la demo 1. A `Math` nunca le hiciste `new`. A `Scanner`, sí: `new Scanner(System.in)`. ¿Por qué? Porque `Scanner` no es una caja de herramientas, es un **objeto**: cada `Scanner` lee de su propio lugar y recuerda por dónde va. Esa diferencia es toda la demo 1.

Rómpelo

Cambio	Lo que dice <code>javac</code> o <code>java</code>
Quita el <code>Notas.</code> de <code>Notas.promedio(notas)</code>	<code>cannot find symbol ... method promedio(double[]) ... location: class Paso10Clases</code> : lo busca en tu clase, no en <code>Notas</code>
Escribe <code>notas.promedio(notas)</code> , con minúscula	<code>cannot find symbol ... location: variable notas of type double[]</code> : lo busca en el arreglo
En <code>Notas.java</code> , cambia <code>public</code> por <code>private</code> en <code>promedio</code>	<code>promedio(double[]) has private access in Notas</code> (un adelanto de la demo 2)
Ejecuta <code>java Notas</code>	<code>Error: Main method not found in class Notas</code>

Si vienes de Python

En Python esto no necesita una clase: basta con un archivo `notas.py` lleno de funciones, y el archivo **es** el contenedor.

```
# notas.py
def promedio(notas):
    return sum(notas) / len(notas)

# otro archivo
import notas
notas.promedio([5.5, 6.2])
```

En Java no existen funciones sueltas: **todo** vive en una clase. Así que la clase hace el papel del módulo. Y no hay `import`: Java ve solo las clases de la misma carpeta (eso lo trabaja la [hoja 00](#)).

Ahora tú

- Agrega `Notas.minima(double[] notas)` y úsala.

- Crea una clase `Convertor` con `celsiusAFahrenheit(double c) ( c * 9 / 5 + 32 )` y `kmAMillas(double km) ( km * 0.621371 )`, y un programa `Paso10Convertor` que la use. ¿Cómo tiene que llamarse el archivo de cada una?
- Muda el `esPar` del paso 9 a una clase `Numeros`.

✓ **Al terminar el paso 10**, tu código tiene que hacer lo mismo que `Notas.java` + `Paso10Clases.java`. Compruébalo con `javac Notas.java Paso10Clases.java` y `java Paso10Clases`, y si algo no calza, compara los dos archivos línea por línea.

 **Ejercicios del paso 10:** paso10/ejercicios.md

Paso 11 - Validaciones

Objetivo: que el programa no se caiga ni acepte basura, escriba lo que escriba el usuario.

En el paso 4, escribir `veinte` en la edad botaba el programa. En el paso 5, una nota de 9 se aceptaba feliz. Hay tres cosas que revisar:

Qué se valida	Ejemplo	Si no se valida
El tipo	que la edad sea un número	<code>InputMismatchException</code> y el programa muere
El rango	que la nota vaya de 1.0 a 7.0	un promedio de 8.2 que nadie nota
El formato	que el correo termine en <code>@duocuc.cl</code>	datos basura guardados para siempre

Vamos a juntar todo en una clase `Validador`, con **dos familias** de métodos:

- los que **preguntan** (`es...`, `esta...`): reciben un dato y responden `true` o `false`. No leen ni imprimen nada.
- los que **insisten** (`leer...`): piden el dato por teclado **hasta que sirva**, y lo devuelven. Usan a los de arriba para decidir.

11.1 Los que preguntan

Crea `Validador.java` (con el `import`, que lo vas a necesitar en el 11.2):

```

import java.util.Scanner;

public class Validador {

    public static boolean estaEnRango(double valor, double min, double max) {
        return valor >= min && valor <= max;
    }

    public static boolean esCorreoDuoc(String correo) {
        String dominio = "@duocuc.cl";
        return correo.endsWith(dominio)
            && correo.length() > dominio.length() // algo antes del @
            && !correo.contains(" ");
    }
}

```

Y `Paso11Validar.java`, con el Scanner:

```

import java.util.Scanner;

public class Paso11Validar {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println(Validador.estaEnRango(5.5, 1.0, 7.0));
        System.out.println(Validador.estaEnRango(9.0, 1.0, 7.0));
        System.out.println(Validador.esCorreoDuoc("ana.perez@duocuc.cl"));
        System.out.println(Validador.esCorreoDuoc("ana.perez@gmail.com"));
        System.out.println(Validador.esCorreoDuoc("@duocuc.cl"));
        System.out.println();

        sc.close();
    }
}

```

```

javac Validador.java Paso11Validar.java
java Paso11Validar

```

```
true
false
true
false
false
```

Un `String` trae métodos para revisarlo por dentro:

Método	Pregunta
<code>correo.endsWith("@duocuc.cl")</code>	¿termina con esto?
<code>correo.startsWith("ana")</code>	¿empieza con esto?
<code>correo.contains(" ")</code>	¿tiene esto en alguna parte?
<code>correo.length()</code>	¿cuántos caracteres tiene?
<code>correo.isEmpty()</code>	¿está vacío?
<code>correo.trim()</code>	(no pregunta: devuelve el texto sin espacios en los bordes)

Estos métodos no tocan el teclado. Por eso sirven en cualquier parte: con datos que vienen del usuario, de un archivo o escritos en el código, como aquí.

11.2 Leer un texto que no esté vacío

Agrega a `Validador`:

```
public static String leerTexto(Scanner sc, String pregunta) {
    System.out.print(pregunta);
    String texto = sc.nextLine().trim();
    while (texto.isEmpty()) {
        System.out.println("    No puede quedar vacio.");
        System.out.print(pregunta);
        texto = sc.nextLine().trim();
    }
    return texto;
}
```

Es el `while` que valida del paso 6, metido en un método. El `trim()` hace que un nombre escrito solo con espacios también cuente como vacío.

11.3 Leer un entero sin que el programa se caiga

Esta es la importante. Agrega a `Validador`:

```
public static int leerEntero(Scanner sc, String pregunta, int min, int max) {
    while (true) {
        System.out.print(pregunta);
        if (!sc.hasNextInt()) {
            System.out.println("    Eso no es un numero entero.");
            sc.nextLine();                // bota lo que escribio
        } else {
            int valor = sc.nextInt();
            sc.nextLine();                // se come el Enter
            if (estaEnRango(valor, min, max)) {
                return valor;            // la unica salida del while
            }
            System.out.println("    Tiene que ir de " + min + " a " + max + ".");
        }
    }
}
```

- `sc.hasNextInt()` mira lo que escribió el usuario **sin sacarlo**, y responde: ¿esto es un entero? Así se pregunta **antes** de leer, y `nextInt()` solo se llama cuando no se va a caer.
- Si no es un entero, `sc.nextLine()` **bota** lo que escribió, para que la próxima vuelta mire algo nuevo.
- Si es un entero, se lee, se come el Enter, y se revisa el rango con `estaEnRango`: un método de `Validador` usando otro.
- `while (true)` no es un error: es un ciclo que no termina por la condición, sino por el `return`. Y al `return` solo se llega con un dato bueno.

11.4 Leer un decimal: escríbelo tú

`leerDecimal(Scanner sc, String pregunta, double min, double max)` es **el mismo método** que `leerEntero`, cambiando lo que haya que cambiar. Escríbelo sin mirar, y después compáralo con [paso11/referencia/Validador.java](#).

Pista: `Scanner` también tiene `hasNextDouble()`.

11.5 Usarlos

Agrega al `main`, antes del `sc.close()`:

```

String nombre = Validador.leerTexto(sc, "Nombre: ");
int edad = Validador.leerEntero(sc, "Edad: ", 15, 99);
double nota = Validador.leerDecimal(sc, "Nota: ", 1.0, 7.0);

String correo = Validador.leerTexto(sc, "Correo DUOC: ");
while (!Validador.esCorreoDuoc(correo)) {
    System.out.println("    Tiene que terminar en @duocuc.cl");
    correo = Validador.leerTexto(sc, "Correo DUOC: ");
}

System.out.println();
System.out.printf("%s, %d años, nota %.1f, %s\n", nombre, edad, nota,
correo);

```

Ahora **intenta botarlo**: deja el nombre vacío, escribe `abc` en la edad, una nota de 9, un correo de Gmail.

```

Nombre:
    No puede quedar vacío.
Nombre: Ana Perez
Edad: abc
    Eso no es un número entero.
Edad: 12
    Tiene que ir de 15 a 99.
Edad: 20
Nota: 9
    Tiene que ir de 1.0 a 7.0.
Nota: 5.5
Correo DUOC: ana@gmail.com
    Tiene que terminar en @duocuc.cl
Correo DUOC: ana.perez@duocuc.cl

Ana Perez, 20 años, nota 5.5, ana.perez@duocuc.cl

```

Dos cosas para mirar con calma:

- Ya no hay ningún `sc.nextLine()` suelto en el `main`. La trampa del Enter del paso 4 la resuelve `Validador` una vez, y nunca más te preocupas.
- El correo combina las dos familias: `leerTexto` **insiste** hasta que no esté vacío, y `esCorreoDuoc` **pregunta** si el formato sirve.

Si tu PC está en español, recuerda la coma (paso 4): `5.5` ahora no bota el programa, pero te va a decir `Eso no es un numero.` Escribe `5,5`.

Rómpelo

Cambio	Qué pasa
En <code>leerEntero</code> , borra el <code>sc.nextLine()</code> que "bota lo que escribió", y escribe <code>abc</code>	<code>Eso no es un numero entero.</code> sin parar. <code>hasNextInt()</code> mira sin sacar: el <code>abc</code> se queda ahí para siempre. Ctrl + C.
En <code>estaEnRango</code> , cambia <code>&&</code> por <code> </code>	<code>estaEnRango(9.0, 1.0, 7.0)</code> da <code>true</code> : con <code> </code> basta que se cumpla una (9 es mayor que 1)
En <code>esCorreoDuoc</code> , borra la línea del <code>length()</code>	<code>@duocuc.cl</code> , sin nada antes, pasa como correo válido

Validar antes de que se caiga. Aquí el programa pregunta **antes** de leer, y nunca llega a caerse. En la demo 10 vas a ver la otra forma: dejar que se caiga y atrapar la caída con `try-catch`. Las dos sirven, pero la de hoy es la que conviene siempre que se pueda.

Si vienes de Python

```
while True:
    texto = input("Edad: ")
    if texto.isdigit():
        edad = int(texto)
        if 15 <= edad <= 99:
            break
        print("    Tiene que ir de 15 a 99.")
    else:
        print("    Eso no es un numero entero.")
```

Python	Java
<code>texto.isdigit()</code>	<code>sc.hasNextInt()</code> (antes de leer)
<code>15 <= edad <= 99</code>	<code>edad >= 15 && edad <= 99</code> (no se pueden encadenar)
<code>texto.strip()</code>	<code>texto.trim()</code>
<code>texto.endsWith("@duocuc.cl")</code>	<code>texto.endsWith("@duocuc.cl")</code>
<code>" " in texto</code>	<code>texto.contains(" ")</code>

Python	Java
<code>len(texto) == 0</code>	<code>texto.isEmpty()</code>

Ahora tú

- Agrega `static boolean leerSiNo(Scanner sc, String pregunta)`: devuelve `true` con `s` o `S`, `false` con `n` o `N`, e insiste con cualquier otra cosa.
- Usa `leerEntero` en el menú del paso 6. Ahora escribirle `abc` no lo bota.
- **El RUT.** Escribe `esRutValido(String rut)`: que tenga un guion, y que antes del guion haya solo dígitos (pista: `rut.indexOf('-')`, `rut.charAt(i)` y `Character.isDigit(c)`). Extra: busca cómo se calcula el dígito verificador (módulo 11) y compruébalo también.

✓ **Al terminar el paso 11**, tu código tiene que hacer lo mismo que `Validador.java` + `Paso11Validar.java`. Compruébalo con `javac Validador.java Paso11Validar.java` y `java Paso11Validar`, y si algo no calza, compara los dos archivos línea por línea.

 **Ejercicios del paso 11:** [paso11/ejercicios.md](#)

Paso 12 - Sobrecarga: un nombre, varias versiones

Objetivo: escribir varios métodos que se llaman **igual**, y entender cómo Java decide cuál usar. Es la primera forma de **polimorfismo**.

12.1 Tres áreas, un nombre

Crea `Geometria.java`:

```

public class Geometria {

    // Circulo: un dato
    public static double area(double radio) {
        return Math.PI * radio * radio;
    }

    // Rectangulo: dos datos
    public static double area(double base, double altura) {
        return base * altura;
    }

    // Triangulo, por sus tres lados (formula de Heron)
    public static double area(double a, double b, double c) {
        double s = (a + b + c) / 2;
        return Math.sqrt(s * (s - a) * (s - b) * (s - c));
    }
}

```

Y `Paso12Sobrecarga.java`:

```

public class Paso12Sobrecarga {

    public static void main(String[] args) {
        System.out.printf("Circulo de radio 2:      %6.2f%n", Geometria.area(2));
        System.out.printf("Rectangulo de 3 x 4:      %6.2f%n", Geometria.area(3,
4));
        System.out.printf("Triangulo de lados 3-4-5: %6.2f%n", Geometria.area(3, 4,
5));
    }
}

```

```

javac Geometria.java Paso12Sobrecarga.java
java Paso12Sobrecarga

```

```

Circulo de radio 2:      12.57
Rectangulo de 3 x 4:      12.00
Triangulo de lados 3-4-5:  6.00

```

Tres métodos `area` en la misma clase, y Java no se confunde. Para Java, un método no se identifica solo por su nombre, sino por su nombre **y sus parámetros**: eso se llama la **firma**.

Llamada	Firma que calza	Figura
<code>area(2)</code>	<code>area(double)</code>	círculo
<code>area(3, 4)</code>	<code>area(double, double)</code>	rectángulo
<code>area(3, 4, 5)</code>	<code>area(double, double, double)</code>	triángulo

Java elige **al compilar**, mirando cuántos datos le pasaste y de qué tipo. (Le pasaste enteros y los métodos piden `double`: no hay problema, un `int` cabe en un `double`.)

12.2 En vez de valores por defecto

Agrega a `Paso12Sobrecarga`, **arriba del `main`**:

```
static void linea() {
    linea(30);
}

static void linea(int largo) {
    linea(largo, '-');
}

static void linea(int largo, char simbolo) {
    for (int i = 0; i < largo; i++) {
        System.out.print(simbolo);
    }
    System.out.println();
}
```

Y al final del `main`:

```
linea();
linea(10);
linea(10, '*');
```

```
-----
-----
*****
```

Mira el truco: la versión sin parámetros llama a la de uno, que llama a la de dos. El código que dibuja está escrito **una sola vez**; las otras dos solo rellenan lo que falta. Así se hace en Java lo que en Python se hace con valores por defecto.

12.3 Por tipo

La cantidad no es lo único que distingue: también el **tipo**. Agrega estos tres métodos **fuera del `main`**, debajo de los `línea`:

```
static void describir(int n) {
    System.out.println(n + " es un entero");
}

static void describir(double n) {
    System.out.println(n + " es un decimal");
}

static void describir(String texto) {
    System.out.println "\"" + texto + "\" es un texto de " + texto.length() + "
letras");
}
```

Y llámalos al final del `main`:

```
describir(7);
describir(7.0);
describir("siete");
línea();
```

```
7 es un entero
7.0 es un decimal
"siete" es un texto de 5 letras
-----
```

12.4 La sobrecarga que usas desde el paso 1

Estas cuatro líneas cierran el `main`:

```
System.out.println(7);
System.out.println(7.0);
System.out.println('7');
System.out.println(true);
```

¿Nunca te preguntaste cómo `println` acepta **cualquier cosa**? No es magia: `println` tiene **diez versiones**, una para `int`, otra para `double`, otra para `char`, para `boolean`, para `String`... Desde el paso 1 que usas sobrecarga sin saberlo. `Math.max` también: hay una para `int` y otra para `double`.

¿Y el polimorfismo?

Polimorfismo significa "muchas formas": un mismo nombre que se comporta distinto según con qué lo uses. `area` es un nombre con tres formas.

En Java hay dos maneras de lograrlo:

	Quién decide	Cuándo	Dónde se ve
Sobrecarga (hoy)	el compilador, mirando los parámetros	antes de ejecutar	aquí, y vuelve en la demo 6
Sobrescritura	el programa, mirando el objeto	mientras corre	demo 4

La segunda es la importante de la programación orientada a objetos: un `for` que le pide el `area()` a cada figura sin saber si es un círculo o un rectángulo. Son las **mismas figuras** de este paso; en la demo 4 cada una será una clase.

Rómpelo

Cambio	Qué pasa
Agrega a <code>Geometria</code> un cuadrado: <code>public static double area(int lado) { return lado * lado; }</code>	Compila sin aviso, y el "círculo de radio 2" ahora dice 4.00 . El <code>2</code> es <code>int</code> , y Java prefiere la versión cuyo tipo calza exacto . <code>Geometria.area(2.0)</code> sigue siendo el círculo.
Agrega <code>public static int area(double radio)</code>	<code>method area(double) is already defined in class Geometria</code> : el tipo de retorno no cuenta , solo los parámetros
Llama a <code>Geometria.area(1, 2, 3, 4)</code>	<code>no suitable method found for area(int,int,int,int)</code> , y abajo la lista de las tres versiones que sí existen
Llama a <code>Geometria.area(1, 1, 5)</code>	<code>NaN (Not a Number)</code> : ese triángulo no existe, y nadie lo validó (paso 11)

La primera es la lección del paso: sobrecargar con `int` y `double` para hacer cosas **distintas** es una trampa. Las versiones de un método sobrecargado deberían hacer **lo mismo** con datos distintos. Deshazlo.

Si vienes de Python

Python **no tiene sobrecarga**. Si escribes dos funciones con el mismo nombre, la segunda **reemplaza** a la primera:

```
def area(radio):
    return 3.14159 * radio * radio

def area(base, altura):      # la de arriba dejo de existir
    return base * altura

area(2)      # TypeError: area() missing 1 required positional argument: 'altura'
```

Lo de `línea` Python lo resuelve con **una** función y valores por defecto:

```
def linea(largo=30, simbolo="-"):
    print(simbolo * largo)
```

Java no tiene valores por defecto, así que escribe una versión por cada combinación. Es el mismo contraste de la demo 1 con los constructores: Java resuelve con **muchas versiones**, Python con **una flexible**.

Ahora tú

- Agrega a `Geometria` un `perimetro` sobrecargado igual que `area`: círculo `2 * Math.PI * radio`, rectángulo `2 * (base + altura)`, triángulo `a + b + c`.
- Agrega `esTriangulo(double a, double b, double c)`: cada lado tiene que ser menor que la suma de los otros dos. Úsalo antes de calcular el área.
- **Promedio ponderado**. Agrega a `Notas` un segundo `promedio(double[] notas, double[] pesos)`. Con notas `{5.0, 6.0, 4.0}` y pesos `{0.3, 0.3, 0.4}` tiene que dar 4.9. Ahora `Notas.promedio` tiene dos formas.

✓ **Al terminar el paso 12**, tu código tiene que hacer lo mismo que `Geometria.java` + `Paso12Sobrecarga.java`. Compruébalo con `javac Geometria.java Paso12Sobrecarga.java` y `java Paso12Sobrecarga`, y si algo no calza, compara los dos archivos línea por línea.

 **Ejercicios del paso 12:** paso12/ejercicios.md

Paso 13 - Todo junto: calculadora de notas

Objetivo: un programa completo que use los doce pasos anteriores, y las dos clases que ya escribiste.

Lo que tiene que hacer

1. Pedir el nombre de un alumno y cuántas notas tiene (de 1 a 10).
2. Pedir cada nota, sin aceptar notas fuera de 1.0 a 7.0.
3. Mostrar las notas, el promedio y si aprobó (con 4.0). Si reprobó, cuánto le faltó.
4. Preguntar si quiere ingresar otro alumno, y repetir todo si dice o .
5. **No caerse con nada:** ni letras, ni nombres vacíos, ni notas de 9.

```
Nombre del alumno: Ana Perez
Cuántas notas tiene? abc
    Eso no es un número entero.
Cuántas notas tiene? 0
    Tiene que ir de 1 a 10.
Cuántas notas tiene? 3
Nota 1: 5.5
Nota 2: 8
    Tiene que ir de 1.0 a 7.0.
Nota 2: 6.2
Nota 3: 3.1
```

```
===== Ana Perez =====
```

```
Nota 1: 5.5
Nota 2: 6.2
Nota 3: 3.1
Promedio: 4.9
APROBADO
```

```
Otro alumno? (s/n): s
Nombre del alumno: Luis Soto
Cuántas notas tiene? 2
Nota 1: 3.5
Nota 2: 3.9
```

```
===== Luis Soto =====
```

```
Nota 1: 3.5
Nota 2: 3.9
Promedio: 3.7
REPROBADO, le faltan 0.3
```

```
Otro alumno? (s/n): N
Listo.
```

`Notas.java` y `Validador.java` **ya están en tu carpeta**: no se copian, se usan. Eso es lo que se gana con haberlas separado.

Intenta cada etapa antes de abrir el código. Crea `Paso13Notas.java`.

Etapas Etapa 1 - Un alumno, a mano

Sin clases y sin validar: pide el nombre y la cantidad, crea un `double[]` de ese largo, llénalo con un `for` e imprime el promedio.

Usa: pasos 4, 7 y 8. Acuérdate de la trampa del Enter.

▼ Ver el código de la etapa 1

```
import java.util.Scanner;

public class Paso13Notas {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Nombre del alumno: ");
        String nombre = sc.nextLine();

        System.out.print("Cuántas notas tiene? ");
        int cantidad = sc.nextInt();

        double[] notas = new double[cantidad];
        for (int i = 0; i < notas.length; i++) {
            System.out.print("Nota " + (i + 1) + ": ");
            notas[i] = sc.nextDouble();
        }

        double suma = 0;
        for (double n : notas) {
            suma += n;
        }
        System.out.printf("Promedio de %s: %.1f\n", nombre, suma /
notas.length);

        sc.close();
    }
}
```

Escríbele `abc` en la cantidad: se cae. Las etapas 2 y 3 lo arreglan.

Etapa 2 - El informe, con `Notas`

Crea un método `static void informe(String nombre, double[] notas)` arriba del `main`, que imprima el recuadro con las notas, el promedio y si aprobó. El cálculo **no** lo escribas: pídeselo a `Notas`. Borra del `main` la suma y el `printf`, y en su lugar llama a `informe(nombre, notas);`.

Usa: pasos 9 y 10.

▼ Ver el código de la etapa 2

```
static void informe(String nombre, double[] notas) {
    System.out.println();
    System.out.println("==== " + nombre + " ====");
    for (int i = 0; i < notas.length; i++) {
        System.out.printf("  Nota %d: %.1f%n", i + 1, notas[i]);
    }

    double p = Notas.promedio(notas);
    System.out.printf("  Promedio: %.1f%n", p);
    if (Notas.aprobo(p)) {
        System.out.println("  APROBADO");
    } else {
        System.out.printf("  REPROBADO, le faltan %.1f%n",
Notas.NOTA_APROBACION - p);
    }
    System.out.println();
}
```

Etapa 3 - Blindarlo, con **Validador**

Cambia cada `sc.nextLine()`, `sc.nextInt()` y `sc.nextDouble()` del `main` por el `validador.leer...` que corresponda. Para el rango de las notas, usa las constantes de `Notas`. Después intenta botarlo.

Usa: paso 11.

▼ Ver el código de la etapa 3

```
String nombre = Validador.leerTexto(sc, "Nombre del alumno: ");
int cantidad = Validador.leerEntero(sc, "Cuántas notas tiene? ", 1,
10);

double[] notas = new double[cantidad];
for (int i = 0; i < notas.length; i++) {
    notas[i] = Validador.leerDecimal(sc, "Nota " + (i + 1) + ": ",
        Notas.NOTA_MINIMA, Notas.NOTA_MAXIMA);
}

informe(nombre, notas);
```

Fíjate: desaparecieron todos los `print` de las preguntas (ahora la pregunta se le pasa a `Validador`), y no hay que comerse ningún Enter.

Etapa 4 - Varios alumnos

Envuelve todo en un `do-while` que se repita mientras la respuesta a `Otro alumno? (s/n):` sea `s` o `S`.

Usa: pasos 5 (`equalsIgnoreCase`) y 6 (`do-while`). ¿Dónde tiene que estar declarada la variable de la respuesta para que el `while` del final la vea?

▼ Ver el main terminado

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String otro;

    do {
        String nombre = Validador.leerTexto(sc, "Nombre del alumno: ");
        int cantidad = Validador.leerEntero(sc, "Cuántas notas tiene? ", 1,
10);

        double[] notas = new double[cantidad];
        for (int i = 0; i < notas.length; i++) {
            notas[i] = Validador.leerDecimal(sc, "Nota " + (i + 1) + ": ",
                Notas.NOTA_MINIMA, Notas.NOTA_MAXIMA);
        }

        informe(nombre, notas);

        otro = Validador.leerTexto(sc, "Otro alumno? (s/n): ");
    } while (otro.equalsIgnoreCase("s"));

    System.out.println("Listo.");
    sc.close();
}
```

El programa completo está en [paso13/referencia/Paso13Notas.java](#).

Compara el largo de tu `main` con el de la etapa 1: hace mucho más y es casi igual de corto. Todo lo difícil vive en `Notas` y `Validador`, escrito una vez.

Ahora tú

- **El alumno que aprobó y reprobó a la vez.** Ingresa un alumno con dos notas: `3.9` y `4.0`. El programa dice `Promedio: 4.0` y, justo abajo, `REPROBADO, le faltan 0.0`. ¿Cómo puede ser? Arréglalo. (Pista: el promedio real es 3.95. ¿Quién redondea y quién no? Pasos 2 y 3. Y el arreglo va en `Notas`, así sirve para todos los programas que la usen.)
- Muestra en el informe la nota máxima y cuántas notas rojas tuvo: los métodos ya existen en `Notas`.
- Al salir, muestra cuántos alumnos aprobaron y cuántos reprobaron en total.

- Si hiciste los desafíos del paso 11 y 12: usa `leerSiNo` para la pregunta final, y el promedio ponderado para un ramo con notas de 30%, 30% y 40%.

✓ **Al terminar el paso 13**, tu código tiene que hacer lo mismo que `Notas.java` + `Validador.java` + `Paso13Notas.java`. Compruébalo con `javac Notas.java Validador.java Paso13Notas.java` y `java Paso13Notas`, y si algo no calza, compara los dos archivos línea por línea.

 **Ejercicios del paso 13:** [paso13/ejercicios.md](#)

Chuleta: Python → Java

	Python	Java
Punto de partida	el archivo, de arriba abajo	<code>public static void main(String[] args)</code>
Ejecutar	<code>python3 archivo.py</code>	<code>javac Archivo.java</code> y <code>java Archivo</code>
Fin de instrucción	salto de línea	<code>;</code>
Bloques	sangría	<code>{ }</code>
Imprimir	<code>print(x)</code>	<code>System.out.println(x);</code>
Formato	<code>f"{x:.2f}"</code>	<code>System.out.printf("%.2f%n", x);</code>
Variable	<code>edad = 20</code>	<code>int edad = 20;</code>
Constante	<code>IVA = 0.19</code> (costumbre)	<code>final double IVA = 0.19;</code> (obligado)
Leer texto	<code>input()</code>	<code>sc.nextLine()</code>
Leer entero	<code>int(input())</code>	<code>sc.nextInt()</code> (+ <code>sc.nextLine()</code> después)
Condición	<code>if x > 3:</code> / <code>elif</code>	<code>if (x > 3) {</code> / <code>else if</code>
Lógicos	<code>and</code> <code>or</code> <code>not</code>	<code>&&</code> <code> </code> <code>!</code>
Comparar textos	<code>a == b</code>	<code>a.equals(b)</code>
Contar	<code>for i in range(5):</code>	<code>for (int i = 0; i < 5; i++) {</code>
Recorrer	<code>for n in notas:</code>	<code>for (double n : notas) {</code>
Lista	<code>[1, 2, 3]</code>	<code>ArrayList</code> (o arreglo, de largo fijo)

	Python	Java
Largo	<code>len(x)</code>	<code>x.length</code> / <code>x.length()</code> / <code>x.size()</code>
Función	<code>def f(x):</code>	<code>static int f(int x) {</code>
Agrupar funciones	un módulo <code>notas.py</code>	una clase con métodos <code>static</code>
Usarlas desde otro archivo	<code>import notas</code> y <code>notas.promedio(x)</code>	<code>Notas.promedio(x)</code> , sin <code>import</code> si está en la carpeta
Mismo nombre, otros parámetros	la segunda función reemplaza a la primera	sobrecarga: conviven las dos
Parámetros opcionales	<code>def linea(largo=30):</code>	una versión sobrecargada por combinación
¿Es un número?	<code>texto.isdigit()</code>	<code>sc.hasNextInt()</code>
Rango	<code>1 <= x <= 7</code>	<code>x >= 1 && x <= 7</code>
Potencia	<code>2 ** 10</code>	<code>Math.pow(2, 10)</code>
División entera	<code>7 // 2</code>	<code>7 / 2</code> (entre enteros)

¿Y ahora?

Si llegaste hasta aquí, sabes todo lo que necesitas para empezar con objetos. Lo que sigue:

1. [EJERCICIOS-00-ARCHIVOS.md](#): 15 minutos sobre la regla "una clase, un archivo", que empezaste a usar en el paso 10.
2. **Demo 1**: tu primera clase de la que se crean **objetos** con `new`, como `Scanner`, y tus primeros métodos **sin** `static`.